FRDM-RW612 Experiments Execution Guide

IoT Lab (EC2601-21) | Board: **NXP FRDM-RW612** | Workspace: `D:\NXP_Programs`

1. Overview of Experiments

Experiment	Project Folder	Lab Exp #	Primary Test Tools	Key Deliverable
BLE Communication	`frdmrw612_wireless_uart`	Exp 8	IoT Toolbox or nRF Connect (Smartphone)	Bidirectional Serial over GATT
MQTT Pub / Sub	`wifi_mqtt`	Exp 6	Mosquitto & MQTT Explorer (PC)	Publish/Subscribe via lwIP stack
Matter over Wi-Fi / Thread	`frdm_rw612_light_switch_otbr_combo`	Exp 5	NXP Chip-tool (Android App)	BLE Commissioning & Dual Endpoint Control

> **Common Hardware & Serial Terminal Setup:**
> • Connect the FRDM-RW612 to the PC via a USB-C cable to the on-board MCU-Link debug port.
> • Open a Serial Terminal (VS Code Serial Monitor / Tera Term / PuTTY) at: **115200 baud, 8 data bits, no parity, 1 stop bit, no flow control**.
> • J-Link software is installed at: `C:\Program Files\SEGGER\JLink_V952\JLink.exe`.

2. Experiment A: BLE Communication (wireless_uart)

What it does: The board implements a custom GATT "Wireless UART" service that emulates a serial stream over BLE. By default it advertises as `NXP_WU`. A short press of user button SW1 makes it scan as a central; a long press returns it to peripheral mode.

2.1 Build Status & Binaries

- Compiler flag updated in `armgcc/flags.cmake` (`-Og`) to resolve GCC 14 register constraints in crypto modules.
- Firmware built at: `frdmrw612_wireless_uart/armgcc/flash_debug/wireless_uart.srec` (and `.elf`, `.hex`, `sdk20-app.bin`).

2.2 Flashing Procedure

1. Open terminal or Command Prompt in `D:\NXP_Programs\frdmrw612_wireless_uart`.
2. Execute the automated flashing batch script:

   ```
   flash_jlink.bat
   ```

   (This runs J-Link with `commands_script`: reset, halt, loadfile `wireless_uart.srec`, reset, go).
3. Press **SW1** or reset the board.

2.3 Verification Steps

1. **Check Serial Output:** The terminal should print:

   ```
   BLE Wireless Uart demo start...
   Bluetooth initialized
   Advertising successfully started
   ```

2. **Connect via Smartphone (Peripheral Test):**
   - Open **NXP IoT Toolbox** → Select **Wireless UART** (or open **nRF Connect**).
   - Tap **NXP_WU** to connect. *Accept the pairing popup in your phone's notification bar.*
   - **Phone → Board:** Type a message on the phone; verify it appears in your PC serial terminal.
   - **Board → Phone:** Type characters in the serial terminal; confirm they display on the phone screen.

3. Experiment B: MQTT Publish/Subscribe (wifi_mqtt)

What it does: The board connects to a 2.4 GHz Wi-Fi AP, connects to an MQTT broker, subscribes to `lwip_topic/#` (QoS 0) and `lwip_other/#` (QoS 1), and publishes *"message from board"* to `lwip_topic/100`.

3.1 Set Up Local Mosquitto Broker

1. Find your PC's IPv4 address: Run `ipconfig` in terminal (e.g. `192.168.1.50`).
2. Start the local Mosquitto broker:

```
cd D:\NXP_Programs\wifi_mqtt
start_mosquitto.bat
```

(Uses `mosquitto.conf` with port 1883 and `allow_anonymous true`).

3.2 Configure Wi-Fi & Broker in wifi_mqtt.c

Open `D:\NXP_Programs\wifi_mqtt\wifi_mqtt.c` and verify/set lines 33–49:

```
#define AP_SSID "Your_2.4GHz_SSID"
#define AP_PASSWORD "Your_WiFi_Password"
#define EXAMPLE_MQTT_SERVER_HOST "192.168.1.50"  /* PC IPv4 or "test.mosquitto.org" */
#define EXAMPLE_MQTT_SERVER_PORT 1883
```

3.3 Build & Flash

1. In VS Code or terminal, build preset `flash_debug`.
2. Run the flashing script:

```
flash_jlink.bat
```

3.4 Verification Steps

1. **Open MQTT Explorer on PC:**
   - Host: `localhost` (or PC IP), Port: `1883`. Click **Connect**.
   - Add topic subscription: `lwip_topic/#`.
2. **Verify Terminal Output:**

```
[i] Successfully initialized Wi-Fi module
Connecting as client to ssid: Your_SSID
[i] Connected to Wi-Fi
IPv4 Address : 192.168.x.x
Connecting to MQTT broker at 192.168.1.50...
MQTT client "nxp_xxxx" connected.
Subscribed to the topic "lwip_topic/#".
Going to publish to the topic "lwip_topic/100"...
Received 18 bytes from the topic "lwip_topic/100": "message from board"
Published to the topic "lwip_topic/100".
```

3. **Subscriber Test:** In MQTT Explorer, publish a custom message to `lwip_topic/test` → verify it appears in the serial terminal console.
4. **HTTP Note:** For the HTTP requirement, browse to the board's IP address (using the `wifi_httpsrv` firmware).

4. Experiment C: Matter over Wi-Fi / Thread (Light Switch OTBR)

What it does: The board functions as a Matter accessory over Wi-Fi with an integrated OpenThread Border Router (OTBR). Endpoint 1 is a dimmable Light Bulb, and Endpoint 2 is a Light Switch. It is commissioned via Bluetooth using the NXP Chip-tool app on Android.

4.1 Firmware Preparation

- **Route 1 (Prebuilt):** Place `chip-rw61x-light-switch-combo-example.srec` (or `.hex`) directly into `D:\NXP_Programs\frdm_rw612_light_switch_otbr_combo`.
- **Route 2 (Build on Linux/WSL):** Run `./build_matter_linux.sh` which clones Matter v1.4.0.0, overlays files, and runs `gn gen` & `ninja`.

4.2 Flashing Procedure

1. Execute the flashing script in `D:\NXP_Programs\frdm_rw612_light_switch_otbr_combo`:

   ```
   flash_jlink.bat
   ```

   (Uses `commands_script` to flash `chip-rw61x-light-switch-combo-example.srec` via J-Link).

4.3 Commissioning & Control (NXP Chip-tool Android App)

1. **Access Point Requirement:** Connect Android phone to a **2.4 GHz WPA2** network (WPA3 is not supported; ensure IPv6 DHCP is disabled on AP).
2. **Factory Reset Board:** Hold **SW2** for 3 seconds (or type `matterfactoryreset` in serial console).
3. **Pair in App:**
   - Enable phone Bluetooth. Open **NXP Matter Chip-tool**.
   - Tap **Pair New Device (WiFi Device)** → Select **Manual Entry**.
   - Enter **Discriminator:** `3840` and **PIN:** `20202021`.
   - Enter your Wi-Fi SSID and Password when prompted.
4. **Control the Light:**
   - After pairing completes, name the device and toggle the light **ON / OFF** in the app.
   - Press physical buttons on the FRDM-RW612 to toggle the local bulb and observe status update in the app.
5. **CLI Commands (Optional):**
   - `otcli state` → View Thread role (Leader / Router).
   - `otcli dataset active` → Inspect Thread channel, PAN ID, and network key.

5. Lab Report Screenshots Checklist

Screenshot Type	BLE (Exp 8)	MQTT (Exp 6)	Matter (Exp 5)
1. Project Workspace	VS Code project view	VS Code project view	Folder / build directory
2. Build Success	Build Output (0 errors)	Build Output (0 errors)	Ninja build completion
3. Flashing Log	J-Link flash log	J-Link flash log	J-Link flash log
4. Serial Output	Advertising & TX/RX log	Wi-Fi IP & Pub/Sub log	Boot & Commissioning log

Screenshot Type	BLE (Exp 8)	MQTT (Exp 6)	Matter (Exp 5)
5. App / Tool Output	IoT Toolbox chat window	MQTT Explorer message tree	Chip-tool app ON/OFF UI